



PUC Minas

**Leonardo Vieira Machado
693447**

Teste de Software

Análise de Eficácia de Testes com Teste de Mutação

Belo Horizonte 2026

1. Análise Inicial

Ao observar os resultados das ferramentas, notamos uma diferença significativa entre a eficácia teórica e a eficácia real dos testes:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	85.41	58.82	100	98.64	
operacoes.js	85.41	58.82	100	98.64	112
Test Suites: 1 passed, 1 total					
Tests: 50 passed, 50 total					
Snapshots: 0 total					
Time: 1.308 s					

Cobertura de Código (Jest): 85.41% de Statements e 100% de Funções. Embora pareça alta, a cobertura de linhas (**98.64%**) indica apenas que o código foi executado, não que foi validado corretamente.

Ran 1.18 tests per mutant on average.							
File	% Mutation score		# killed	# timeout	# survived	# no cov	# errors
	total	covered					
All files	73.71	78.11	153	4	44	12	0
operacoes.js	73.71	78.11	153	4	44	12	0
14:03:42 (16504) INFO HtmlReporter Your report can be found at: file:///C:/Users/leovi/Desktop							
14:03:42 (16504) INFO MutationTestExecutor Done in 23 seconds.							

Podemos verificar que a análise inicial do Stryker indica uma cobertura de apenas 73,71%

A discrepância ocorre porque a cobertura de código é uma métrica de **quantidade** (o teste passou por aqui?), enquanto o teste de mutação mede a **qualidade** (o teste falha se o comportamento mudar?). Temos muitos testes que executam o "caminho feliz", mas não verificam condições de erro, limites ou lógica booleana, permitindo que mutantes sobrevivam mesmo em linhas cobertas.

2. Análise de Mutantes Críticos

Escolhemos três mutantes sobreviventes que revelam fraquezas estruturais na suíte original:

Mutante A: Lógica Booleana em isPar

- **Arquivo:** src/operacoes.js: Ln 43
- **Mutação:** "n % 2 === 0" alteração para "true"

- **Por que não morreu?** O teste original apenas chamou `isPar(100)`, que retorna `true`. Como o mutante força o retorno a ser sempre `true`, o teste continua passando. O teste falhou em verificar o caso negativo (um número ímpar).

Mutante B: Condicional de Erro em `raizQuadrada`

- **Arquivo:** `src/operacoes.js`: Ln 12
- **Mutação:** “`if (n < 0)`” alterada para “`if (false)`”
- **Por que não morreu?** O teste original apenas verificou `raizQuadrada(16)`. Ele nunca enviou um número negativo para disparar a cláusula `throw`. Portanto, se removermos a proteção contra números negativos, o teste não percebe.

Mutante C: Cálculo da Mediana (Paridade)

- **Arquivo:** `src/operacoes.js`: Ln 111
- **Mutação:** “`if (sorted.length % 2 === 0)`” alterada para “`false`”
- **Por que não morreu?** O teste original usou um array de 5 elementos (ímpar). A lógica para arrays de tamanho par nunca foi exercitada, tornando toda a ramificação do “`if`” invisível para a suíte de testes.

3. Solução Implementada

Mutante A: Lógica Booleana em `isPar`

```
test('15. deve retornar true para um número par', () => { expect(isPar(100)).toBe(true); });
test('15.1 deve retornar false para um número ímpar', () => {
  // Mata o mutante de condição que sempre retorna true
  expect(isPar(7)).toBe(false);
});
```

Foi adicionado um teste específico (15.1) que espera o retorno `false` para um número ímpar (ex: 7).

Testes básicos muitas vezes verificam apenas o "caminho feliz" (se um número par retorna `true`). Se um mutante alterasse a lógica da função para sempre retornar `true`, o teste original passaria erroneamente. Ao validar explicitamente o cenário negativo (ímpar), garantimos que a condição lógica `n % 2 === 0` é avaliada corretamente.

Mutante B: Condicional de Erro em `raizQuadrada`

```
test('6. deve calcular a raiz quadrada de um quadrado perfeito', () => { expect(raizQuadrada(16)).toBe(4); });
test('6.1 deve lançar erro ao calcular raiz quadrada de número negativo', () => {
  // Mata os mutantes de condição e cobre o StringLiteral
  expect(() => raizQuadrada(-1)).toThrow(
    'Não é possível calcular a raiz quadrada de um número negativo.',
  );
});
test('6.2 deve calcular a raiz quadrada de zero', () => {
  // Mata mutante de igualdade (n <= 0)
  expect(raizQuadrada(0)).toBe(0);
});
```

Inclusão de verificação para o valor limite zero (6.2) e validação da mensagem de erro (6.1).

Mutantes frequentemente alteram operadores de comparação (de < 0 para $<= 0$). Testar o zero garante que a função permite o cálculo da raiz de zero, "matando" mutantes que tentariam proibir esse valor.

Validar a mensagem exata do erro impede que o código seja alterado para uma mensagem genérica ou vazia, garantindo a integridade da comunicação com o usuário/sistema.

Mutante C: Cálculo da Mediana (Paridade)

```
test('47. deve calcular a mediana de um array ímpar e ordenado', () => { expect(medianaArray([1, 2, 3, 4, 5])).toBe(3); });
test('47.1 deve calcular a mediana de um array ímpar e desordenado', () => {
  // Mata mutante de 'sort' que remove a ordenação (ex: [...numeros])
  expect(medianaArray([5, 3, 1, 4, 2])).toBe(3);
});
test('47.2 deve calcular a mediana de um array par', () => {
  // Mata mutantes no bloco 'if' para arrays pares
  expect(medianaArray([1, 2, 3, 4])).toBe(2.5);
});
test('47.3 deve lançar erro ao calcular mediana de array vazio', () => {
  // Mata os mutantes de condição e cobre o StringLiteral
  expect(() => medianaArray([])).toThrow('Array vazio não possui mediana.');
```

Verificação de arrays desordenados (47.1), arrays com extensão par (47.2) e tratamento de erro para arrays vazios (47.3).

47.1: Um mutante poderia remover ou alterar a linha `.sort()`. O teste com um array desordenado (ex: `[5, 3, 1, 4, 2]`) obriga a função a ordenar os dados para chegar ao resultado correto (3). Sem isso, a função falharia silenciosamente.

47.2: O teste para arrays de tamanho par valida o cálculo da média entre os dois valores centrais, garantindo que o mutante não simplifique a lógica para apenas retornar um único índice.

47.3: Garante que a lógica de proteção para arrays vazios está ativa e disparando o erro correto.

4. Resultados Finais

```

Ran 2.20 tests per mutant on average.
-----|-----|-----|-----|-----|-----|-----|
File    | % Mutation score | # killed | # timeout | # survived | # no cov | # errors |
-----|-----|-----|-----|-----|-----|-----|
All files | 96.71 | 96.71 | 202 | 4 | 7 | 0 | 0 |
operacoes.js | 96.71 | 96.71 | 202 | 4 | 7 | 0 | 0 |
-----|-----|-----|-----|-----|-----|
14:23:17 (5676) INFO HtmlReporter Your report can be found at: file:///C:/Users/leovi/Desktop
14:23:17 (5676) INFO MutationTestExecutor Done in 26 seconds.

```

Após a atualização da suíte com 78 testes (em comparação aos 50 iniciais), os resultados demonstraram um salto qualitativo expressivo:

- **Pontuação de Mutação Final: 96.71%.**
- **Mutantes Mortos: 202** de 213 totais.
- **Cobertura de Mutação: 100%** (todos os 213 mutantes foram cobertos pelos testes, restando apenas 7 sobreviventes que são mutantes equivalentes).

A melhoria na suíte é comprovada pela eliminação de quase todos os mutantes, atingindo um estado onde a lógica do sistema está quase totalmente blindada contra alterações acidentais.

5. Conclusão

O teste de mutação provou ser uma ferramenta superior à simples cobertura de código. Ele transformou uma suíte de testes "passiva" em uma ferramenta de "verificação ativa". Ao forçar falhas no código fonte e exigir que os testes as detectem, garantimos que a suíte não apenas execute as linhas de código, mas que efetivamente valide as regras de negócio e os limites do sistema, resultando em um software muito mais robusto e confiável. Contudo, deve ser balanceado o custo da execução dessa abordagem. O maior investimento de tempo necessário para sua aplicação pode não ser capaz de justificar sua utilização dependendo do negócio.